Semester Application Security Testing and Debugging Runbook

Version: 0.1 draft | Issued: 2026-10-06 | Owner: Engineering/security (appoint)
Status: Authorized-test baseline, not an executed assessment

1. Rules of engagement

Record authorization, exact targets, environments, accounts, allowed techniques, time window, rate limits, exclusions, contacts, stop conditions, and data cleanup. Default to isolated staging and synthetic records. Never scan arbitrary third-party integrations or institutional systems without their authorization. Stop on production impact, unexpected sensitive data, or scope escape; notify the incident contact and preserve minimal redacted evidence.

2. Prepare and baseline

Pin commit/build/configuration; enumerate URLs, APIs, roles, tenants, database objects, uploads, jobs, integrations, and trust boundaries. Verify tool versions and rule-set versions. Establish healthy baseline, snapshot test fixtures, and rollback capability. Inspect repository scripts before executing them; do not assume command names from documentation. Previously discussed supabase/check.sh must be located and reviewed before use.

3. Vulnerability scanning sequence

1. Secret scanning: current tree, relevant Git history, bundles, source maps, docs, CI output, and artifacts. Candidate tools: Gitleaks or equivalent. Treat suppressions as reviewed exceptions, not broad allowlists.
2. Dependency/SCA: inspect lockfiles and transitive dependencies with supported package tooling or a maintained scanner; record runtime/dev/build scope, exploitability, reachability, and advisory version. Generate an SBOM when supported.
3. SAST: use maintained TypeScript/JavaScript and SQL rules (for example Semgrep/CodeQL as applicable). Review unsafe HTML, injections, authorization gaps, SSRF paths, unsafe redirects, secrets, and dangerous shell/process usage.
4. Configuration/IaC: inspect CI permissions, deployment exposure, environment separation, headers/CORS, storage access, database grants/policies, and integration scopes.
5. Authenticated staging DAST: use an approved scanner such as OWASP ZAP with separate role contexts. Exclude destructive operations unless explicitly approved. Verify authentication remains active during the scan; unauthenticated coverage is not authenticated coverage.
6. Manually validate findings, deduplicate, record false-positive rationale, and compare scan coverage to the asset inventory.

Tools are proposed examples, not confirmation they are installed or configured. Store sanitized machine-readable results and reviewer decisions. Failed or incomplete scans cannot be labeled clean.

4. Manual code review checklist

Trace an untrusted input from browser/webhook/import to validation, authn, authz, DB/storage/worker, and output. Validate authorization at the final trusted execution boundary and for every object ID, not just the route. Check caller-derived identity, tenant selection, grants, service-role shortcuts, and mass assignment. Review rich text and HTML output, parameterized queries, URL destinations/redirects, upload validation, size limits, token storage, session lifecycle, exception/log contents, and cryptographic API use. Inspect retries, idempotency, compensation, race conditions, event ordering, and revocation freshness. Review dependencies and workflow scripts added by the change. Include direct API/RPC access rather than accepting a hidden button as protection.

5. Core test cases

ID	Setup/action	Expected result	Required observation
T-AUTH-01	Expired/forged token or invalid issuer/audience	Denied	No privileged work or data
T-AUTH-02	Reuse session after disable/revoke	Denied within defined revocation bound	API, DB, realtime, cache behavior
T-AC-01	Student A requests B's private ID	No private rows	HTTP/body plus side effects
T-AC-02	Tenant T1 supplies T2 ID	Denied on every equivalent path	RPC/export/storage/realtime
T-AC-03	Client modifies roles or ownership	Rejected	No changed grants/row ownership
T-INJ-01	Test metacharacters in query/filter fields	Treated as data	No query alteration or errors leaking internals
T-XSS-01	Harmless marker payload in supported text surfaces	No script execution	Stored/reflected/DOM contexts
T-SSRF-01	URL-import feature targets approved internal sentinel	Blocked per destination policy	No forbidden network request
T-FILE-01	Mismatched type/oversized upload	Rejected safely	No publicly readable orphan
T-API-01	Excess requests at approved staging rate	Enforced quota/limit	No runaway cost or bypass
T-LOG-01	Fail auth/action using synthetic secret marker	Redacted	App/DB/CI/error tooling logs
T-WF-01	Repeat/out-of-order event and interrupted step	One authorized effect, recoverable state	Durable state/idempotency/audit
T-WF-02	Revoke permission mid-workflow	No new unauthorized steps	Fresh decision/compensation policy

ID	Setup/action	Expected result	Required observation
T-REC-01	Isolated restore and rollback drill	Integrity and access controls restored	RPO/RTO and restored-grant checks
T-AI-01	If AI present, untrusted text requests tool override	No policy bypass or unauthorized write	Tool scope, approval and audit

Add feature-specific tests. Test success and rejection paths. Assertion includes actual state, not only HTTP status. The RLS document defines AC-01 through AC-16; these are mandatory for affected migrations.

6. Secure debugging protocol

1. Classify: routine defect or possible security incident. Contain credible active exposure before deep diagnosis.
2. Capture minimal sanitized evidence: correlation ID, UTC times, role/tenant, commit/deploy, config/migration version, expected/actual result.
3. Preserve evidence access-controlled; never paste production records, tokens, keys, or raw auth headers into tickets/chat/AI prompts.
4. Reproduce with synthetic fixtures in isolated staging. Avoid attaching debuggers or verbose sensitive logs to production without explicit operational approval.
5. Trace layers: UI -> gateway/API -> validated identity -> capability/tenant -> DB policy/RPC -> integration/job -> audit event.
6. Identify root cause, equivalent vulnerable paths, timing/race behavior, and impact; do not stop at symptom suppression.
7. Implement smallest safe change, add regression test, review independently, run full affected suite and verify rollback/migration compatibility.
8. Deploy with approval, verify authorized and denied paths, monitor, then remove temporary diagnostics.

A safe log records identifiers/categories only as needed; never a whole request object. Do not change multiple production variables simultaneously without a tracked plan.

7. Finding and remediation workflow

Required fields: ID, discovery time, source, affected asset/version/environment, reproduction steps, expected/actual result, minimal sanitized proof, severity rationale, impact, exploitability, owner, containment, proposed fix, due date, exception ID if any, retest result, deployed version, closure approver.

States: NEW -> TRIAGED -> CONTAINED (if needed) -> IN_PROGRESS -> READY_FOR_RETEST -> VERIFIED -> DEPLOYED -> CLOSED. FALSE_POSITIVE and ACCEPTED_EXCEPTION require documented approvals; exceptions expire. Reopen on failed retest. Independence means a second qualified reviewer where feasible; disclose limitations in a small team.

Critical: immediate containment; target fix 24 hours where feasible. High: 7 days. Medium: 30 days. Low: 90 days. These proposed internal targets require approval and do not replace legal

deadlines. Confirmed sensitive-data exposure, privilege escalation, and active secret compromise are release blockers regardless of scanner score.

8. Retest and closure

Reproduce the original case, test sibling endpoints/RPCs, verify all affected roles/tenants, validate no unauthorized side effects, run positive cases and broader regression, test fresh/upgraded schema, and validate exact production release configuration. Attach sanitized evidence and link the test to a permanent CI guard when practical. Close only after deployment verification and risk-owner sign-off; a merged PR alone is not closure.

9. Evidence and reporting

Report scope, dates, authorized targets, commit/deploy, tool/rule versions, tested roles and resources, limitations, findings by severity, blocked controls, exceptions, remediation owners/dates, and evidence links. Never infer complete coverage from a scanner pass. Keep sensitive proof private and publish only approved aggregate statements.

10. Incident tabletop

Scenario: a student sees another institution's document. Walk through report intake, containment, evidence preservation, object/RPC/storage path review, token/key decisions, affected population analysis, institutional/legal communication, safe fix, retest, and restoration. Measure time to contain and record unresolved decisions. Drill backup restore and privilege revocation separately.